



MEGA BACKEND · SOLUTION ARCHITECT PORTFOLIO

SHADOWLEAGUE NEXUS

Architecture & Delivery Blueprint · v2.1

REVISED EDITION

51 checkpoints · 1,282 central hours · internal Spring platform · BOM and starters · Artifactory · portfolio launch

Prepared for Antonio · deliberate learning at 4 technical hours per week

Sandbox payments only · no real money · no production claims

26 August 2026 · Europe/Madrid

What changed in v2.1

CONCLUSION

The previous blueprint covered microservices, Kafka, security and operations, but it did not explicitly train the design of a reusable internal platform. This edition closes that gap without turning the project into a premature library collection.

Scope changes

- S00 now creates the public repository, Apache-2.0 license, minimum governance and architectural vision.
- New S23 delivers a BOM, Kafka auto-configuration and starter, test support, service template and Artifactory publication.
- Former S23-S48 checkpoints move to S24-S49; no functional scope is removed.
- New S50 packages the release, case study, demo, social preview, LinkedIn Featured entry and interview narrative.
- The central estimate rises from 1,212 to 1,282 hours; the macro range becomes 950-1,700 hours.

What remains unchanged

- Start with a modular monolith and extract services only when evidence justifies the cost.
- Every extracted service owns its data; there is no global transaction across databases.
- Distributed consistency uses Saga, Outbox/Inbox, idempotency, compensation and reconciliation.
- Code, ADRs, tests, diagrams and runbooks remain mandatory evidence.

What the blueprint fixes—and what you decide

RULE

The blueprint fixes the problem, constraints, evidence and risks. You choose and defend the concrete solution. A quest without a decision, test or verifiable artifact does not build senior judgement.

Category	Responsibility
Fixed	Business flow, sandbox only, 4 h/week, Definition of Done, security, no secrets and exact money.
You decide	Bounded contexts, model, contract, locking, retries, versioning, extraction and rollback.
You measure	Latency, contention, duplicates, error budget, cost, capacity and actual completion time.
You document	Alternatives, trade-offs, consequences, reversibility and signals that would change the decision.
You may reject	Any technology or pattern that protects no invariant or cannot justify its operational cost.

Mandatory questions before implementation

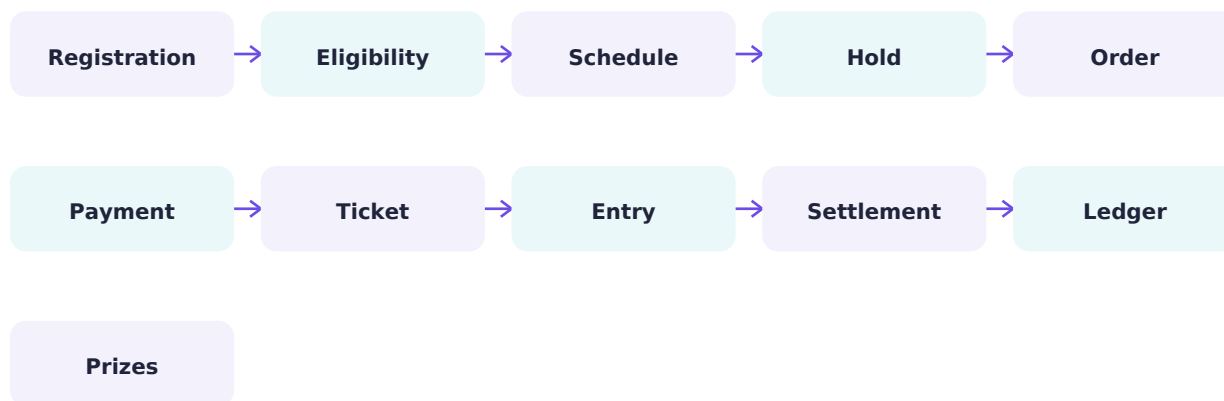
- Which functional or quality driver makes this component necessary?
- Where is the source of truth, and which invariant does it protect?
- Which alternatives were rejected, and why?
- How does it fail, how is it observed, and how is it reconciled?
- Who upgrades, versions and operates it?
- How is it deployed, migrated, rolled back and retired without breaking consumers?

ShadowLeague Nexus at a glance

Problem

A multi-tenant platform coordinating federations, clubs, athletes, fans, venues, sponsors and suppliers across the full sporting and financial lifecycle of a competition.

Main flow



Macro scenario: Matchday Settlement Saga

- Prevent overselling the last seat with a temporary hold and one transactional authority.
- Create an idempotent order and process payment only through a sandbox provider.
- Issue the ticket, balanced ledger entry and notification under one correlation chain.
- Converge after timeout, duplicate, reordering, crash and unknown provider outcome.
- Settle beneficiaries and reconcile provider, orders, ledger and payouts.

PORTFOLIO STANDARD

Success is not a high service count. Success is explaining why each boundary exists, how consistency converges, and which evidence validates the decision.

Evidence expected from a future Solution Architect

Dimension	Expected evidence
Business	Drivers, actors, constraints, priorities, language and measurable outcomes.
Application	DDD, modularity, hexagonal architecture, APIs, events, tests and evolution.
Data	Ownership, local ACID, consistency, ledger, privacy and retention.
Integration	Contracts, idempotency, ordering, Saga, compensation and reconciliation.
Platform	BOM, starters, templates, Artifactory, DevEx, versioning and adoption.
Security	Threat model, identities, secrets, least privilege and supply chain.
Operations	SLOs, observability, capacity, cost, incidents, DR and runbooks.
Communication	ADRs, C4, roadmap, trade-offs, stakeholders, demo and defence.

SENIOR DIFFERENCE

Knowing a pattern is not enough. Connect it to a risk, choose where it belongs, measure the result, and explain when not to use it.

Distribute only when evidence pays for it

THESIS

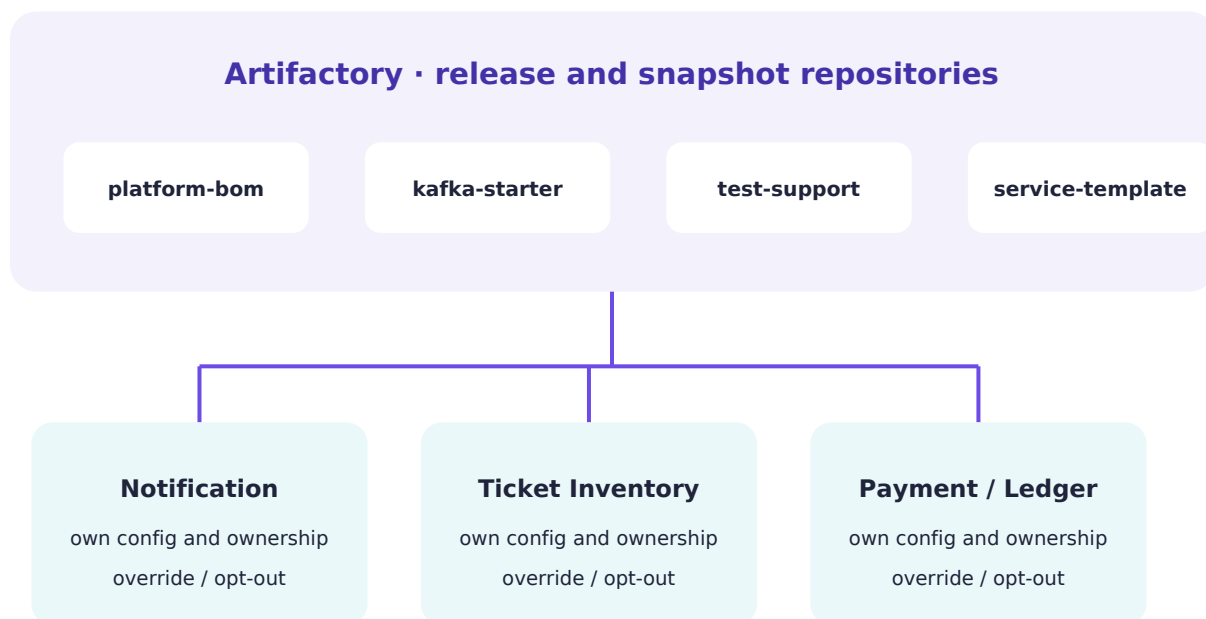
Competition Core begins as a modular monolith. Ticket Inventory, Notification and Payment/Ledger are extracted when contention, blast radius, deployment, security or ownership justify the distributed cost.

Form	Benefit	Cost / risk
Modular monolith	Local ACID, lower operations, faster refactoring	Boundary discipline
Microservice	Independent scale, isolation and deployment	Network, data, observability
Internal library	Technical consistency and less repetition	Version coupling
Template / golden path	Consistent bootstrap without runtime dependency	Drift between copies
Platform service	Central control and specialised operations	Dependency and latency

Extraction signals

- A demonstrably different load or availability profile.
- A proven need for independent deployment and rollback.
- Security, compliance or blast-radius isolation.
- Separate ownership and rate of change.
- A specialised store or operating model with measurable value.

Spring reuse topology



Platform contract

- Provide safe defaults and technical wiring; never own business rules or domain events.
- Publish compatibility, changelog, deprecations and rollback guidance.
- Reduce adoption time without hiding failures or blocking local decisions.
- Validate the product in two services with different configurations before claiming success.

ANTI-PATTERN AVOIDED

A common JAR with shared entities, DTOs and exceptions creates a distributed monolith. The platform shares mechanisms; bounded contexts preserve meaning and autonomy.

What is published—and what stays local

Artifact	Responsibility	Guardrail
platform-bom	Compatible versions	Import through dependency management
kafka-autoconfigure	Beans, properties, metadata	No business listeners
kafka-starter	Convenient entry point	Minimal transitive dependencies
test-support	EmbeddedKafka and technical fixtures	No domain data
service-template	Golden path	Evolvable copy, not a runtime JAR
observability-starter	Only after two adoptions	Correlation without PII

Internal-product policy

- Each artifact has an owner, consumers, support promise and feedback channel.
- Adoption must reduce measurable friction: bootstrap time, defects or duplicated configuration.
- Breaking changes require a major version, migration guide and deprecation window.
- A capability without two consumers or measurable savings remains local.

Artifactory token versus runtime secrets

BUILD / CI

- Scoped, expiring token
- settings.xml or CI secret store
- Resolve and publish artifacts
- Secret scanning and SBOM



NEVER REUSE

RUNTIME

- Workload identity
- Separate Kafka credentials
- Secret manager / env / Kubernetes
- Rotation and least privilege

Acceptance controls

- The token never appears in Git, logs, images, JARs, application variables or copyable documentation.
- CI separates dependency resolution from publication and applies narrow scopes and repositories.
- Revoking the Artifactory token does not affect an already deployed service.
- Rotating Kafka credentials does not require republishing the library.
- The threat model covers exfiltration, dependency confusion and artifact tampering.

app.kafka.error-handler

Property	Type	Intent
enabled	boolean	Enable or disable the common policy
dlt.suffix	string	Error-destination suffix
dlt.useSamePartition	boolean	Partition preservation
backoff.mode	fixed exponential	Explicit retry strategy
backoff.interval	duration	Initial delay
backoff.maxInterval	duration	Exponential ceiling
backoff.retries	integer	Bounded attempts
backoff.multiplier	decimal	Growth factor
nonRetryable	class list	Failures that must not be retried

PRECEDENCE

Safe starter defaults < service application.yml < environment variables or secrets < documented bean override. Every level must be observable and validated at startup.

Decisions the starter must not hide

- Handler idempotency
- Required ordering
- DLT retention
- Replay authorisation
- Terminal exception policy

Starter Definition of Done

MANDATORY INTEGRATED TEST

One `@EmbeddedKafka` test class must demonstrate bean creation, injection/wiring, blocking retry and DLT/backoff behaviour. It must fail if auto-configuration does not load, a local override loses precedence, or a terminal message misses its expected destination.

Evidence components

- `KafkaTestUtils`, `KafkaTemplate`, `ProducerFactory` and `ConsumerFactory`.
- `KafkaAdmin` and `KafkaListenerEndpointRegistry`.
- `CommonErrorHandler` and `BackOff` configured through properties.
- Two contexts: starter defaults and service override/opt-out.
- Assertions for delivery, retries, DLT, headers and correlation.

Minimum compatibility matrix

Version	Java	Spring	Kafka
Starter 1.x	Java 21	Spring Boot baseline	Kafka client baseline
Compatibility	N-1 when viable	Test matrix	Release notes
Breaking change	Major	Migration guide	Deprecation window

Repository, licensing and minimum governance

File	Purpose
README.md	Problem, scope, architecture, quick start and status
LICENSE	Apache-2.0 at repository root
.gitignore	Java, Maven/Gradle, IDE and local secrets
SECURITY.md	Reporting process, supported versions and no secrets
CONTRIBUTING.md	Branches, commits, tests and review
docs/adr/ADR-0001	Repository, licence and initial decision
PR template	DoD, tests, docs, security and rollback

RECOMMENDED LICENCE

Apache-2.0 is permissive and grants an explicit patent licence. The ADR compares MIT, Apache-2.0 and no licence. This is a portfolio decision, not legal advice.

Exit gate

Public repository with no secrets; versioned vision and scope; C4 system-context diagram; actor map; initial glossary; contribution policies; and the first traceable decision.

51 checkpoints · S00-S50

S00-S15

Foundations, modular monolith, sport domain, inventory, payments, ledger and settlement

370 h through S15

S16-S22

Outbox/Inbox, Kafka, service extractions and Sagas

+174 h

S23

Internal Spring platform, BOM, starters and Artifactory

48 h

S24-S33

Marketplace, suppliers, payouts, CQRS, search, cache, analytics and privacy

Ecosystem

S34-S43

Security, resilience, OTel/SLO, capacity, Kubernetes, IaC, CI/CD and DR

Operations

S44-S49

E2E, cost, fitness functions, contracts, demo and defence

Validation

S50

Release, case study, demo and LinkedIn

16 h

No AI time discount

Work type	Range	Includes
Reading / review	0.5-1.5 h	Notes plus application
Modelling / ADR	1-3 h	Alternatives plus decision
Implementation + tests	2-4 h	Code, debugging and evidence
Vertical slice / infra	4-8 h	2-4 sessions
Work over 8 h	Split it	Keep the full DoD

Milestones

Through	Central	At 4 h/week	Outcome
S08	178 h	45 weeks	Publishable modular core
S15	370 h	93 weeks	Coherent financial MVP
S21	544 h	136 weeks	Saga and extractions
S23	614 h	154 weeks	Adopted internal platform
S50	1,282 h	321 weeks	Complete portfolio launch

MACRO RANGE

950-1,700 hours. The central figure is not a promise; recalibrate it with the median actual/estimated ratio from the last 5-10 comparable quests.

A reviewer's ten-minute journey

Time	What the reviewer can verify
0-1 min	Problem, scope and hybrid-architecture thesis
1-3 min	C4, ADR-001 and domain boundaries
3-5 min	Demo: sandbox purchase and last-seat contention
5-7 min	Timeout/duplicate and Saga convergence
7-8 min	Balanced ledger and reconciliation
8-9 min	Traces, metrics, SLOs and runbooks
9-10 min	Internal starter, compatibility and trade-offs

NARRATIVE

Problem → constraints → simplest decision → extraction signals → internal platform → distributed consistency → operations → measured outcomes and limits.

No inflated claims

- Separate tests, simulations and production experience.
- Never use microservice count as a success metric.
- Publish limitations and next steps.

S00.Q01 · 5 h · 4 sessions

DELIVERABLE

Initial public repository with Apache-2.0, .gitignore, SECURITY.md, CONTRIBUTING.md, pull-request template, ADR-0001, vision README and C4 system-context diagram.

Block	Time	Outcome
Session 1	60-90 min	Repository, licence, ADR-0001 and secret hygiene
Session 2	60-90 min	Problem, purpose and actors
Session 3	50-70 min	Capabilities and out-of-scope
Session 4	75-105 min	C4, trade-offs, review and commit

Essential sources

[GitHub Docs · Creating a new repository](#)

[GitHub Docs · Licensing a repository](#)

[Apache License 2.0](#)

[C4 · System context diagram](#)

S00.Q02 REMAINS LOCKED

Do not mark the checkpoint complete until the full Definition of Done is met. Record partial progress as actual hours, then resume the same quest.

PART I

Product vision and macro use case

A sports ecosystem creates meaningful reasons for payments, reservations, contracts, event spikes, external providers and settlement controls.

Product statement

ShadowLeague Nexus is a multi-tenant platform for federations, clubs, athletes, fans, sponsors, venues and providers. It coordinates a championship from registration through matchday settlement: eligibility, schedules, tickets, inventory holds, orders, sandbox payments, venue access, merchandising, travel, sponsorship milestones, prizes and payout reconciliation.

Why this terrain is portfolio-grade

- Sports creates scheduled demand spikes: ticket drops, registration deadlines and matchday gate traffic.
- Money introduces idempotency, exact arithmetic, auditability, immutable accounting and reconciliation.
- Inventory introduces races, expirations and the need to prove that overselling cannot occur.
- Clubs, federations and sponsors create multi-tenancy, authorization and contractual workflows.
- Travel, notification and payment providers introduce latency, partial failure, retries and anti-corruption layers.
- Search and analytics justify specialized stores without granting them authority.
- A hybrid monolith/microservice shape demonstrates judgment instead of fashion.

Primary actors

Actor	Goal	Critical risk
Federation operator	Publish competitions and enforce rules	Cross-tenant privilege or invalid lifecycle
Club administrator	Enroll teams, buy services, receive settlements	Duplicate enrollment or incorrect balance
Athlete	Maintain profile and prove eligibility	Privacy leak or stale status
Fan	Buy ticket/merchandise and enter venue	Double charge, lost ticket or oversell
Sponsor	Fund measurable milestones	Unauthorized approval or disputed evidence
Venue operator	Manage capacity and validate access	Conflict, offline duplicate or revoked ticket
Provider	Supply payment, travel, shipping or messages	Timeout, duplicate callback or inconsistent status
Finance operator	Reconcile, settle and resolve exceptions	Unbalanced ledger or opaque manual correction
Auditor	Trace every sensitive decision	Mutable history or missing correlation

PART I

The giant end-to-end scenario

1. Competition setup

A federation creates a season, ruleset, divisions and registration window. Published versions become immutable.

2. Club onboarding

A club joins a tenant, assigns administrators and registers teams and athletes.

3. Eligibility

The platform evaluates age, affiliation, suspension and quota rules against a versioned ruleset.

4. Enrollment transaction

A team consumes quota atomically. Concurrent duplicate attempts produce one accepted enrollment.

5. Scheduling

A solver proposes fixtures; the core verifies team and venue collisions before publication.

6. Commercial setup

The venue and club publish price books, ticket types, fees and beneficiary shares.

7. Ticket drop

Thousands of fans browse derived availability; authoritative holds are created under contention.

8. Checkout

An idempotent order snapshots price and creates a sandbox PaymentIntent. Webhooks may arrive late or duplicated.

9. Saga completion

The orchestrator commits inventory, issues a ticket, posts a journal and confirms the order. Failures compensate.

10. Matchday

Gates validate credentials online or in bounded offline mode, then reconcile access decisions.

11. Settlement

A closed window allocates net proceeds to club, venue, platform, tax and sponsor accounts.

12. Payout

Beneficiary payouts run idempotently. Provider, order and ledger totals are reconciled.

13. Ecosystem

Merchandise is fulfilled, travel bookings are coordinated and sponsorship milestones release value.

14. Data products

Search and analytics update asynchronously with visible freshness and replay support.

15. Incident

A payment provider outage opens the circuit, queues work safely and triggers an operator runbook.

16. Audit

A reviewer traces an enrollment, purchase, access decision and settlement from command to event to journal.

PART I

Scope, non-goals and safety boundaries

In scope

- Synthetic multi-tenant sports domain with clearly owned data and test users.
- Stripe test mode or a deterministic fake provider; idempotency and signed webhook verification.
- Local developer platform with containers, then a Kubernetes/IaC demonstration environment.
- A credible hybrid architecture and progressive extraction path.
- Evidence: tests, ADRs, diagrams, traces, dashboards, runbooks and failure demonstrations.

Explicit non-goals

- No real-money operation, card storage, KYC promise, tax/legal advice or regulatory certification.
- No custom identity provider, payment gateway, message broker or database implementation.
- No mobile/web product completeness; a thin admin/demo UI or API client is enough.
- No claim of exactly-once business effects across the whole network.
- No database sharing that allows one extracted service to write another service's tables.
- No microservice extraction without an ADR and a measured reason.

MVP technical slice

The first demonstrable slice is deliberately narrow: federation creates a competition; club enrolls a team; operator publishes one match; fan buys one ticket in test mode; ticket is issued; a balanced journal is posted; the scenario is traceable. Marketplace, travel, payout and analytics follow later without changing the core narrative.

PART I

Functional requirements catalogue

Seventy concrete product requirements define what the architecture must preserve.

Identity and tenancy

ID	Requirement
FR-001	Authenticate users through OIDC and issue tenant-aware authorization context.
FR-002	Support federation admin, club admin, athlete, fan, sponsor, venue operator and auditor roles.
FR-003	Prevent every cross-tenant read and write unless an explicit federation policy allows it.
FR-004	Record privileged actions in an immutable audit stream.
FR-005	Support service identities and key rotation without user credentials.
FR-006	Allow membership validity windows and revocation.
FR-007	Enforce least privilege at endpoint and use-case boundaries.
FR-008	Provide consent/purpose metadata for user-facing communications.
FR-009	Avoid storing payment credentials in ShadowLeague.
FR-010	Provide a sandbox-only operator role for portfolio demonstrations.

Competition and eligibility

ID	Requirement
FR-011	Create competitions, seasons, divisions, phases and rulesets.
FR-012	Publish immutable versions of rules and schedules.
FR-013	Register clubs, teams and athlete profiles.
FR-014	Evaluate eligibility against the exact ruleset version.
FR-015	Reserve quotas atomically during enrollment.
FR-016	Reject duplicate enrollment under concurrent requests.
FR-017	Withdraw enrollment while preserving history.
FR-018	Schedule matches while preventing team and venue collisions.
FR-019	Process controlled reschedule requests after publication.
FR-020	Close a season and freeze authoritative standings inputs.

Ticketing and access

ID	Requirement
FR-021	Publish ticket offers and price books by event and audience segment.
FR-022	Create expiring holds under peak concurrency.
FR-023	Guarantee no seat or quota is oversold.
FR-024	Commit or release holds idempotently.
FR-025	Issue a signed ticket credential only after payment policy succeeds.
FR-026	Revoke, transfer or reissue a credential with audit history.
FR-027	Validate tickets online and in a constrained offline gate mode.
FR-028	Reconcile offline scans and detect conflicting access.
FR-029	Expose bounded inventory availability without promising stale values are reservable.
FR-030	Support controlled event cancellation and bulk refund workflow.

Orders and payments

ID	Requirement
FR-031	Create one logical order per idempotent create request.
FR-032	Snapshot prices, fees, taxes and beneficiary shares.
FR-033	Create one provider payment intent per order/session.
FR-034	Handle asynchronous success, failure, action-required and dispute events.
FR-035	Verify webhook signatures and deduplicate provider events.
FR-036	Support authorization/capture and immediate capture policies.
FR-037	Support partial and full refund with exact arithmetic.
FR-038	Model chargebacks without rewriting historical payment facts.
FR-039	Expose payment status independently from order status while defining convergence rules.
FR-040	Never log PAN, secrets or sensitive provider payload fields.

Ledger and settlement

ID	Requirement
FR-041	Post immutable double-entry journals for every financial movement.
FR-042	Reject any unbalanced journal atomically.
FR-043	Use integer minor units or exact decimal with explicit currency.
FR-044	Maintain accounts for platform, club, venue, tax, sponsor and payable balances.
FR-045	Compute settlement runs from closed source windows.
FR-046	Make settlement runs idempotent and independently auditable.
FR-047	Reconcile provider totals, orders and journals.
FR-048	Create exception cases for discrepancies rather than hiding them.
FR-049	Support prize allocation and payout state machines.
FR-050	Provide trial balance and trace from provider event to postings.

Commercial ecosystem

ID	Requirement
FR-051	Version sponsorship contracts and approval policies.
FR-052	Release sponsorship value only when milestone evidence is approved.
FR-053	Sell merchandise with stock reservation and fulfillment states.
FR-054	Integrate travel quotes and reservations behind anti-corruption adapters.
FR-055	Snapshot provider cancellation policies.
FR-056	Compensate stock, travel and payment operations when a saga fails.
FR-057	Route notifications without making them a success dependency.
FR-058	Index public entities for search without using the index as authority.
FR-059	Feed analytics from replayable events.
FR-060	Export a synthetic audit case for the portfolio demo.

Operations and platform

ID	Requirement
FR-061	Correlate commands, events, traces and provider identifiers.
FR-062	Expose health endpoints with distinct liveness, readiness and startup semantics.
FR-063	Define timeouts before retries and bound every retry budget.
FR-064	Use circuit breakers and bulkheads for unstable dependencies.
FR-065	Provide DLQ inspection and idempotent replay tooling.
FR-066	Measure queue lag, projection freshness and saga age.
FR-067	Define SLOs for browse, hold, checkout, webhook and settlement paths.
FR-068	Back up each source-of-truth datastore and test restore.
FR-069	Provision infrastructure from reviewed code.
FR-070	Produce SBOM, scan artifacts and keep a rollback path.

PART I

Quality attributes and measurable targets

Attribute	Target / architectural consequence
Consistency	No overselling; balanced ledger; idempotent external effects; documented bounded staleness for search and analytics.
Availability	Browse 99.9% monthly target; checkout and holds degrade safely; finance jobs favor correctness over availability.
Latency	Browse p95 < 300 ms; hold p95 < 450 ms at target load; webhook acknowledgement < 2 s after durable inbox write.
Scalability	Ticket inventory and public reads scale separately from Competition Core; partitions preserve aggregate ordering.
Security	OIDC/OAuth2, least privilege, encryption, secret rotation, signed webhooks, no card storage, auditable privilege use.
Recoverability	Per-store RPO/RT0; replayable projections; restore exercises; reconciliation after recovery.
Operability	Traceable sagas, actionable alerts, DLQ/runbooks, capacity dashboards, feature flags and safe migrations.
Maintainability	Module/service boundaries tested; contracts versioned; ADRs capture irreversible or expensive choices.
Portability	Domain core isolated by ports; local stack via containers; cloud services selected through adapters where practical.
Cost	Single-node local developer profile; scale-to-zero/non-production schedules; cost allocation by component.

Quality attribute scenario template

When <source> sends <stimulus> under <environment>, the <artifact> must respond with <measure>, while preserving <invariant>. Every advanced sprint should turn at least one vague goal into this form.

PART II

Domain decomposition and ownership

Each context below has an explicit owner, aggregate boundary, store and consistency promise.

Context map summary

Zone	Shape	Why
Competition Core	Modular monolith	Organization, athlete, competition, enrollment, scheduling and contracts are semantically cohesive and owned by one developer.
Transactional edge	Extracted services	Ticket inventory, payment and ledger have distinct contention, security and blast-radius profiles.
External integration	Small services/adapters	Travel, fulfillment and notification isolate unreliable providers and retry policies.
Derived data	Search and analytics	Different read workloads; rebuildable from authoritative events.
Platform	Managed/open-source capabilities	Identity, broker, telemetry and orchestration are infrastructure, not business domains.

Core ownership rule

Database per extracted service

Ownership is logical before it is physical. Services may share a PostgreSQL cluster in development, but they use separate databases/schemas, credentials, migrations and write authority. Cross-service joins are forbidden in runtime business logic.

PART II - CONTEXT 01

Identity & Access

Authenticate humans and workloads; issue identity context without owning sports profiles.

Architectural shape	Primary aggregates	Authoritative store
External platform capability	Realm, Client, Role, Service account	Keycloak PostgreSQL; secrets in a secret manager

Invariants

- No domain decision depends on display name
- Tenant and subject are explicit in every authenticated command
- Admin privileges are deny-by-default

Command and event surface

Commands	Events
ProvisionUser, AssignRole, RotateClientSecret	UserProvisioned, RoleAssigned, CredentialRotated

Consistency promise

Strong inside IAM; tokens are cached and short lived

Extraction decision

Decision

Treat as platform dependency; do not build a custom auth server

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 02

Tenant & Organization

Own federations, clubs, legal entities, memberships and tenant boundaries.

Architectural shape	Primary aggregates	Authoritative store
Competition Core module	Organization, Club, Membership	PostgreSQL schema in Competition Core

Invariants

- A club belongs to one legal tenant at a time
- Membership date ranges cannot overlap for the same role
- Cross-tenant references are rejected

Command and event surface

Commands	Events
CreateClub, InviteMember, ChangeMembership	ClubCreated, MemberInvited, MembershipChanged

Consistency promise

Local ACID

Extraction decision

Decision

Remain a module until organizational scaling or compliance demands isolation

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 03

Athlete Registry

Manage athlete sports profiles, eligibility facts and club affiliation.

Architectural shape	Primary aggregates	Authoritative store
Competition Core module	Athlete, RegistrationProfile, Affiliation	PostgreSQL

Invariants

- Federation identifier is unique
- Affiliations have non-overlapping validity
- Sensitive health data is out of scope

Command and event surface

Commands	Events
RegisterAthlete, AffiliateAthlete, SuspendAthlete	AthleteRegistered, AthleteAffiliated, AthleteSuspended

Consistency promise

Local ACID; versioned aggregate

Extraction decision

Decision

Stay in core initially; split only for national-scale registry ownership

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 04

Competition Management

Own tournament lifecycle, divisions, phases, rules and standings authority.

Architectural shape	Primary aggregates	Authoritative store
Modular monolith core	Competition, Season, Phase, RuleSet	PostgreSQL

Invariants

- State transitions follow an explicit machine
- Published rules are immutable; revisions create versions
- A season has one active scoring policy

Command and event surface

Commands	Events
CreateCompetition, PublishCompetition, CloseSeason	CompetitionCreated, CompetitionPublished, SeasonClosed

Consistency promise

Local ACID

Extraction decision

Decision

Deliberately remains modular monolith: high semantic cohesion and one owner

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 05

Eligibility & Registration

Evaluate rules and enroll teams/athletes into competitions.

Architectural shape	Primary aggregates	Authoritative store
Competition Core module	Enrollment, EligibilityDecision, Quota	PostgreSQL

Invariants

- Quota cannot become negative
- An athlete cannot be enrolled twice in the same slot
- Decision records the ruleset version

Command and event surface

Commands	Events
EvaluateEligibility, EnrollTeam, WithdrawEnrollment	EligibilityApproved, TeamEnrolled, EnrollmentWithdrawn

Consistency promise

Local ACID with constraints; optimistic version for edits

Extraction decision

Decision

Potential service only if rule evaluation becomes independently owned

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 06

Scheduling & Venue

Create fixtures and reserve venues without time conflicts.

Architectural shape	Primary aggregates	Authoritative store
Competition Core module	FixturePlan, Match, VenueReservation	PostgreSQL; optional solver worker

Invariants

- No venue overlap
- A team cannot play two matches at once
- Published fixtures require a change request

Command and event surface

Commands	Events
GenerateSchedule, ReserveVenue, RescheduleMatch	ScheduleGenerated, VenueReserved, MatchRescheduled

Consistency promise

Local ACID for reservations; async optimization jobs

Extraction decision

Decision

Keep scheduling authority in core; workers may scale independently

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 07

Contracts & Sponsorship

Track commercial contracts, milestones, approvals and entitlements.

Architectural shape	Primary aggregates	Authoritative store
Competition Core module	Contract, Milestone, ApprovalPolicy	PostgreSQL; object storage for documents

Invariants

- Approved versions are immutable
- Segregation of duties for payout milestones
- Entitlements cannot exceed contract limits

Command and event surface

Commands	Events
DraftContract, ApproveMilestone, RevokeEntitlement	ContractActivated, MilestoneApproved, EntitlementRevoked

Consistency promise

Local ACID; optimistic lock on drafts

Extraction decision

Decision

Remain a module; document processing can be separate

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 08

Catalog & Pricing

Define tickets, fees, discounts and merchandise offers.

Architectural shape	Primary aggregates	Authoritative store
Ticketing service capability	Offer, PriceBook, DiscountPolicy	PostgreSQL

Invariants

- Money uses minor units/decimal
- Published price books are versioned
- Discounts cannot make total negative

Command and event surface

Commands	Events
PublishOffer, PriceCart, RetireOffer	OfferPublished, CartPriced, OfferRetired

Consistency promise

Strong per price book; immutable snapshots in orders

Extraction decision

Decision

Extract with ticketing when sale traffic diverges

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 09

Ticket Inventory

Own seats, quotas and expiring holds under high contention.

Architectural shape	Primary aggregates	Authoritative store
Extracted microservice	InventoryPool, Seat, Hold	PostgreSQL source of truth; Redis for short-lived acceleration only

Invariants

- No overselling
- A seat has at most one live hold or issued ticket
- Expired holds release capacity idempotently

Command and event surface

Commands	Events
CreateHold, ExtendHold, CommitHold, ReleaseHold	HoldCreated, HoldExpired, InventoryCommitted

Consistency promise

Atomic SQL/constraints; optimistic retry; selected row locks

Extraction decision

Decision

Extract at sprint 19 because of spikes, contention and failure isolation

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 10

Order

Own carts converted to immutable commercial orders and lifecycle state.

Architectural shape	Primary aggregates	Authoritative store
Competition Core then boundary service	Order, OrderLine, OrderSnapshot	PostgreSQL

Invariants

- One idempotency key maps to one command fingerprint
- Totals equal signed pricing snapshot
- State transition is monotonic except explicit cancellation

Command and event surface

Commands	Events
CreateOrder, ConfirmOrder, CancelOrder	OrderCreated, OrderConfirmed, OrderCancelled

Consistency promise

Local ACID; eventually consistent with payment

Extraction decision

Decision

May remain with Commerce Core until team/release cadence requires extraction

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 11

Payment Orchestration

Isolate provider integration, payment attempts, webhooks and financial state machine.

Architectural shape	Primary aggregates	Authoritative store
Extracted microservice	Payment, Attempt, ProviderEvent	PostgreSQL; provider vault owns card data

Invariants

- Provider event processed once
- One logical payment intent per order
- Retries reuse idempotency strategy

Command and event surface

Commands	Events
AuthorizePayment, CapturePayment, RefundPayment	PaymentAuthorized, PaymentCaptured, PaymentFailed, RefundSucceeded

Consistency promise

Strong locally; webhook-driven eventual consistency

Extraction decision

Decision

Extract because of security, blast radius and deployment independence

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 12

Ledger & Settlement

Record immutable double-entry journals and settle funds among parties.

Architectural shape	Primary aggregates	Authoritative store
Extracted microservice	Journal, Posting, Account, SettlementRun	PostgreSQL with append-only model

Invariants

- Every journal balances to zero
- Posted entries are immutable
- Settlement run is replay-safe and traceable

Command and event surface

Commands	Events
PostJournal, OpenSettlement, FinalizeSettlement	JournalPosted, SettlementOpened, SettlementFinalized

Consistency promise

Serializable/atomic invariants locally; reconciliation across provider

Extraction decision

Decision

Separate with payments but deploy independently when controls justify it

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 13

Access Control at Venue

Validate issued tickets at gates and tolerate intermittent connectivity.

Architectural shape	Primary aggregates	Authoritative store
Edge-oriented service	TicketCredential, GateSession, AccessDecision	Local edge cache plus PostgreSQL service store

Invariants

- Ticket admits according to policy exactly once or controlled re-entry
- Offline decisions are reconciled
- Revocations propagate with bounded delay

Command and event surface

Commands	Events
ValidateTicket, OpenGateSession, ReconcileOfflineScan	AccessGranted, AccessDenied, OfflineScanReconciled

Consistency promise

Bounded staleness at edge; eventual reconciliation

Extraction decision

Decision

Extract for edge deployment and availability profile

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 14

Marketplace & Fulfillment

Sell club merchandise and coordinate stock and shipment providers.

Architectural shape	Primary aggregates	Authoritative store
Microservice candidate	MarketplaceOrder, StockReservation, Shipment	PostgreSQL; object store for labels

Invariants

- Stock cannot be oversold
- Shipment cannot precede paid/approved order
- Cancellation triggers compensations

Command and event surface

Commands	Events
ReserveStock, CreateShipment, CancelFulfillment	StockReserved, ShipmentCreated, FulfillmentCancelled

Consistency promise

Saga across stock, payment and provider

Extraction decision

Decision

Extract when external vendor integrations and ownership diverge

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 15

Travel Booking

Coordinate quotes and reservations with hotel/transport providers.

Architectural shape	Primary aggregates	Authoritative store
Integration microservice	TravelPlan, Quote, Booking	PostgreSQL; provider data snapshot

Invariants

- Expired quotes cannot be booked
- Duplicate provider confirmations collapse to one booking
- Cancellation policy is snapshotted

Command and event surface

Commands	Events
RequestQuotes, BookTravel, CancelBooking	QuotesReceived, TravelBooked, TravelCancelled

Consistency promise

Saga and compensation; external eventual consistency

Extraction decision

Decision

Separate due to unreliable external dependencies

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 16

Notification

Deliver email, push and in-app communications without blocking core flows.

Architectural shape	Primary aggregates	Authoritative store
Extracted microservice	Notification, TemplateVersion, DeliveryAttempt	PostgreSQL or document store; provider APIs

Invariants

- Deduplication key prevents duplicate user-visible message
- Templates are versioned
- PII is minimized in events

Command and event surface

Commands	Events
QueueNotification, RetryDelivery, SuppressRecipient	NotificationQueued, NotificationDelivered, NotificationFailed

Consistency promise

At-least-once delivery with idempotent rendering

Extraction decision

Decision

First extraction: loose coupling and independent failure profile

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 17

Search

Provide full-text and faceted discovery across public entities.

Architectural shape	Primary aggregates	Authoritative store
Derived microservice	SearchDocument, IndexVersion	OpenSearch

Invariants

- Index is never source of truth
- Rebuild does not cause downtime
- Tenant/public visibility is enforced

Command and event surface

Commands	Events
ReindexEntity, SwapIndexAlias, DeleteSearchDocument	EntityIndexed, IndexRebuilt

Consistency promise

Eventual; lag is measured

Extraction decision

Decision

Extracted read capability fed by domain events

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART II - CONTEXT 18

Analytics & Audit

Produce operational analytics and immutable security/business audit evidence.

Architectural shape	Primary aggregates	Authoritative store
Data platform	Fact tables, AuditRecord, DataProduct	ClickHouse for analytics; object storage for cold audit

Invariants

- Audit records are append-only
- Analytics may lag but freshness is visible
- PII access is purpose-limited

Command and event surface

Commands	Events
IngestEvent, RebuildProjection, ExportAuditCase	ProjectionUpdated, AuditCaseExported

Consistency promise

Eventual, replayable, reconciled

Extraction decision

Decision

Independent pipeline; never called synchronously by checkout

Minimum proof

- Domain tests demonstrate every invariant with one positive and one negative case.
- A contract test covers the public command/event surface.
- A trace or log shows correlation without leaking sensitive data.
- The owning database can be restored or rebuilt according to its role.

PART III

Reference architecture and evolution

Runtime topology

Experience/API	Business execution	Data and integration
Mobile/web demo, Admin client, API Gateway/BFF	Competition Core modular monolith; Ticketing; Order; Payment; Ledger; Travel; Notification	PostgreSQL stores, Kafka, Redis, OpenSearch, ClickHouse, object storage
OIDC tokens and tenant context	Synchronous commands for immediate decisions; asynchronous events for propagation	Outbox/CDC or publisher, Inbox, schema registry, telemetry collector

Evolution rather than big bang

Stage	Deployment units	Communication	Data
A. Modular discovery	One Spring Boot deployment + platform containers	In-process ports and domain/application events	One PostgreSQL instance with module schemas
B. Financial hardening	Core + provider sandbox adapters	Transactional boundaries and durable event registry	Ledger/order/payment schemas with strict ownership
C. Selective extraction	Notification, Ticket Inventory, Payment/Ledger	HTTP for commands; Kafka for facts	Database per extracted service
D. Data products	Search, Analytics, edge Access	Events and projections	OpenSearch, ClickHouse, edge cache
E. Operated platform	Kubernetes/cloud demonstration	Resilience, SLO, runbooks	Backups, retention and recovery tests

Extraction scorecard

Score a capability 0-2 for independent scaling, security/compliance, failure isolation, ownership, release cadence, data workload and edge placement. Extraction is considered at 7+ only after a modular boundary and contract already exist. A high score triggers an ADR, not an automatic split.

PART III

Polyglot persistence with purpose

Owner	Store	Data	Reason	Boundary
Competition Core	PostgreSQL	Organizations, athletes, competitions, enrollments, schedules, contracts	Relational invariants and local transactions	Schema first; may share one cluster but isolated schema/credentials
Ticket Inventory	PostgreSQL + Redis	Seats, quotas, holds, committed inventory; hot TTL hints	Constraints and atomic reservation; Redis is not authority	Dedicated database and credentials
Order	PostgreSQL	Orders, lines, snapshots, idempotency records	Exact totals and lifecycle state	Own database after extraction
Payment	PostgreSQL	Payment intents, attempts, provider inbox	Durable state machine and webhook dedupe	Provider stores payment instruments
Ledger	PostgreSQL	Accounts, journals, postings, settlement runs	Serializable/atomic balance invariants	Most restrictive write role; append-only model
Notification	PostgreSQL or MongoDB	Templates, messages, attempts, suppression	Flexible payloads; at-least-once delivery	Choose MongoDB only with ADR and transaction needs understood
Search	OpenSearch	Derived public documents and facets	Search workload; rebuildable	Never source of truth
Analytics	ClickHouse	Event facts and aggregates	Columnar scans and retention tiers	Replay from durable events/object store
Caching	Redis	Cache entries, rate counters, short-lived coordination hints	TTL and atomic counters	No financial truth or irreplaceable state
Documents	Object storage	Contracts, exports, evidence, audit bundles	Large immutable objects	Metadata and hashes remain in owner database
Event backbone	Kafka/Redpanda local	Domain and integration events	Ordered partitions, replay and consumer isolation	Not a replacement for owner database

PART III

Transactions, consistency and concurrency

First principle

Use one local ACID transaction for one aggregate or tightly coupled invariant. Across service-owned databases, use explicit state machines, Sagas, idempotency and reconciliation. Do not pretend a network call participates in a local @Transactional boundary.

Consistency matrix

Use case	Authority	Model	Concurrency/idempotency	Published fact
Enroll team	Eligibility + quota in Competition Core	Strong/local ACID	Unique constraint + version; retry conflict	EnrollmentAccepted
Hold seat	Ticket Inventory	Strong for reservation	Atomic conditional write / row lock only where measured	HoldCreated
Create order	Order	Strong per order	Idempotency table + command hash	OrderCreated
Authorize/capture	Payment + provider	Eventual external	Provider idempotency + durable webhook inbox	PaymentCaptured
Issue ticket	Ticket Inventory	Eventual after payment	Saga command idempotency + unique order line	TicketIssued
Post journal	Ledger	Strong/local ACID	Balanced journal constraint + unique business key	JournalPosted
Notify fan	Notification	At-least-once	Deduplication key; retries/DLQ	NotificationDelivered
Update search	Search	Eventual	Versioned upsert; lag metric	SearchDocumentUpdated
Analytics fact	Analytics	Eventual/replayable	Event id dedupe; watermark	ProjectionUpdated
Settlement	Ledger + provider payout	Saga + reconciliation	Run idempotency, compensating journal, exception case	SettlementFinalized

Lock selection guide

Mechanism	Use it for	Avoid when	Required test
Optimistic @Version	Human edits, contracts, profiles, low-conflict aggregates	Hot ticket counters with high collision	Two writers; loser receives conflict and retries/merges safely
Unique/check constraint	Duplicate enrollment, business key, journal balance support	Rule cannot be expressed locally	Concurrent insert proves invariant
Atomic conditional update	Decrement available capacity if enough remains	Several rows require ordered locking	Load test shows no negative inventory
SELECT FOR UPDATE	Short critical row set with measured contention	Slow network call inside transaction	Timeout/deadlock behavior and lock order
SERIALIZABLE	Rare correctness-critical transaction with retry policy	High-volume path without capacity testing	Serialization failure is retried safely
Redis lock	Best-effort singleton jobs where duplicate is harmless/idempotent	Financial integrity or sole oversell defense	Expiry, owner token and duplicate execution

PART III

Outbox, Inbox and event delivery

Producer protocol

- Inside one database transaction, update the aggregate and insert an outbox record containing event id, aggregate id/version, type, timestamp, correlation and payload.
- A relay publishes committed outbox records. Ordering is preserved per aggregate key; global ordering is not promised.
- Publication may repeat. The producer marks delivery metadata for operations, not for business exactly-once claims.

Consumer protocol

- Persist event id in an Inbox/processed table within the same local transaction as the consumer's business effect.
- Reject incompatible schema before business execution and route poison messages to a DLQ with diagnostic context.
- Retries use bounded exponential backoff with jitter. Replay tooling is authorization-protected and audited.

Event envelope

Field	Rule
eventId	Globally unique; deduplication identity
eventType	Domain.integration-event.vN naming with explicit major version
occurredAt	Business occurrence in UTC; not broker arrival
producer	Owning service/module and version
aggregateId/version	Ordering and stale-event defense
correlationId	One business journey across commands/events
causationId	Immediate command or event that caused this event
tenantId	Routing/policy metadata; never trust without authorization
payload	Minimal fact, no secrets and minimized PII

PART III

The Matchday Settlement Saga

The capstone transaction spans inventory, order, provider payment, ticket issuance, ledger and later settlement.

#	Participant	Forward action	Success	Compensation	Failure examples
1	Order	Create order from signed price snapshot	OrderCreated	Cancel order	Duplicate request / stale quote
2	Ticket Inventory	Create expiring hold	HoldCreated	Release hold	No capacity / concurrent winner
3	Payment	Create/confirm sandbox PaymentIntent	PaymentAuthorized or Captured	Cancel/refund provider intent	Timeout / action required / decline
4	Ticket Inventory	Commit hold and issue credential	TicketIssued	Revoke ticket and restore inventory by policy	Hold expired / duplicate command
5	Ledger	Post sale journal	JournalPosted	Post reversing journal	Unbalanced entry / duplicate business key
6	Order	Confirm commercial order	OrderConfirmed	Cancel with explicit reason	Out-of-order event
7	Notification	Send confirmation	NotificationDelivered	No financial compensation	Provider unavailable / duplicate event
8	Access	Validate at gate later	AccessGranted	Revoke credential	Offline conflict / revoked ticket
9	Ledger	Open matchday settlement window	SettlementOpened	Abort run before payout	Late event / mismatch
10	Ledger + Payout	Allocate and pay beneficiaries	SettlementFinalized	Compensating journal + exception	Provider payout failure / partial completion

Orchestrator state

The orchestrator persists saga id, order id, current state, deadline, participant command ids, completed steps, compensation status, last error and attempt counters. State transitions are conditional on the expected current state, making repeated commands and out-of-order replies safe.

Timeout policy

- A timeout means unknown, not failed. Query provider/participant state before compensating an operation with uncertain outcome.
- Each command has a business deadline and a technical retry budget. Do not retry card declines or validation failures.
- Compensations are business actions and may fail; they have their own states, retries and operator escalation.

Irreversibility boundary

Before capture, cancellation can usually release the hold and cancel the intent. After capture and ticket issuance, compensation is a refund plus revocation and reversing journal, not a rollback. After venue access, refund policy may require manual review. The saga records which boundary was crossed.

PART III

Saga failure test matrix

Injection	Expected state	Invariant	Evidence
Duplicate create-order request	One order	Idempotency key maps to one command fingerprint	Two responses and one row
Hold service times out before reply	Unknown then reconciled	No second live hold for same request	Status query and trace
Payment decline	Order payment_failed; hold released	No journal/ticket	State and events
Payment response lost after success	Provider queried; capture recognized	No duplicate charge	Provider id and idempotency evidence
Duplicate payment webhook	One local transition	One business effect	Inbox uniqueness
Webhook arrives before API response	State converges	Transition ordering rules	Out-of-order test
Hold expires during customer action	Payment resolved by policy; no invalid ticket	Inventory authority wins	Compensation trace
Ticket issue command duplicated	One credential	Unique order line/command id	Database proof
Ledger unavailable	Saga waits/retries; order not falsely final	Captured money remains traceable	Alert and saga age
Unbalanced journal command	Rejected atomically	Debits equal credits	Constraint/test
Notification provider down	Checkout remains successful	Notification is not financial dependency	DLQ/retry
Broker unavailable after DB commit	Outbox retains event	No lost committed fact	Relay recovery
Consumer crashes after local write	Duplicate delivery is harmless	Inbox and effect atomic	Crash test
Messages reordered	Stale version ignored/parked	Aggregate version monotonic	Reorder test
Settlement run repeated	Same result	Run business key unique	Hash/report comparison
Payout partly succeeds	Exception and compensating accounting	Ledger describes truth	Reconciliation case
Database failover	Retries bounded; no duplicate effect	Transaction outcome reconciled	Failover trace
Clock skew	Expiry uses authority clock/DB time	No client-clock trust	Skew simulation
Redis unavailable	Slower safe path	Source-of-truth integrity unchanged	Fallback metrics
Search unavailable	Checkout unaffected	Derived store not critical	Dependency map

PART III

API surface and contract rules

Competition Core

Method / path	Contract expectation
POST /v1/organizations	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/clubs	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/athletes	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/athletes?cursor=	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/competitions	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/competitions/{id}/publish	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/competitions/{id}/enrollments	Idempotency, authorization, validation, Problem Details and correlation documented
DELETE /v1/enrollments/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/schedules:generate	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/matches/{id}:reschedule	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/contracts	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/contracts/{id}/milestones/{mid}:approve	Idempotency, authorization, validation, Problem Details and correlation documented

Ticketing

Method / path	Contract expectation
GET /v1/events/{id}/offers	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/events/{id}/availability	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/holds	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/holds/{id}:extend	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/holds/{id}:commit	Idempotency, authorization, validation, Problem Details and correlation documented
DELETE /v1/holds/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/tickets/{id}:revoke	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/access:validate	Idempotency, authorization, validation, Problem Details and correlation documented

Order & Payment

Method / path	Contract expectation
POST /v1/orders	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/orders/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/orders/{id}:cancel	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/payments	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/payments/{id}:confirm	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/payments/{id}:capture	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/payments/{id}/refunds	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/provider/stripe/webhooks	Idempotency, authorization, validation, Problem Details and correlation documented

Ledger & Settlement

Method / path	Contract expectation
POST /v1/journals	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/journals/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/accounts/{id}/balance	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/settlements	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/settlements/{id}:finalize	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/reconciliation-runs/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/payouts	Idempotency, authorization, validation, Problem Details and correlation documented

Marketplace & Travel

Method / path	Contract expectation
POST /v1/stock-reservations	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/shipments	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/travel/quotes	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/travel/bookings	Idempotency, authorization, validation, Problem Details and correlation documented
DELETE /v1/travel/bookings/{id}	Idempotency, authorization, validation, Problem Details and correlation documented

Operations

Method / path	Contract expectation
GET /v1/sagas/{id}	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/dlq/{messageId}:replay	Idempotency, authorization, validation, Problem Details and correlation documented
GET /v1/reconciliation/exceptions	Idempotency, authorization, validation, Problem Details and correlation documented
POST /v1/reconciliation/exceptions/{id}:resolve	Idempotency, authorization, validation, Problem Details and correlation documented
GET /actuator/health/readiness	Idempotency, authorization, validation, Problem Details and correlation documented
GET /actuator/health/liveness	Idempotency, authorization, validation, Problem Details and correlation documented

Contract rules

Commands use idempotency keys when externally repeatable. Resources expose stable identifiers and explicit state. Pagination is cursor-based for volatile feeds and Page/Slice decisions are justified for administrative lists. Errors use a consistent problem format without leaking internals.

PART III

Integration event catalogue

Event	Owner	Meaning	Consumers
tenant.organization-created.v1	Tenant	Organization created	Identity, Audit
registry.athlete-registered.v1	Registry	Athlete profile accepted	Eligibility, Search, Audit
competition.published.v1	Competition	Competition becomes public	Search, Notification, Analytics
competition.ruleset-revised.v1	Competition	New rules version	Eligibility, Audit
registration.eligibility-approved.v1	Registration	Decision approved	Enrollment, Notification
registration.team-enrolled.v1	Registration	Quota consumed	Scheduling, Billing, Analytics
registration.enrollment-withdrawn.v1	Registration	Enrollment cancelled	Scheduling, Billing
schedule.generated.v1	Scheduling	Draft fixtures created	Venue, Admin UI
schedule.match-published.v1	Scheduling	Match visible	Ticketing, Search, Notification
schedule.match-rescheduled.v1	Scheduling	Published time changed	Ticketing, Travel, Notification
contract.activated.v1	Contracts	Commercial contract active	Entitlements, Audit
sponsor.milestone-approved.v1	Contracts	Milestone approved	Ledger, Notification
catalog.offer-published.v1	Catalog	Sellable offer available	Search, Ticketing
inventory.hold-created.v1	Inventory	Capacity reserved	Order, Analytics
inventory.hold-expired.v1	Inventory	Capacity released	Order, Analytics
inventory.committed.v1	Inventory	Capacity consumed	Ticket, Analytics
order.created.v1	Order	Commercial order created	Saga, Analytics
order.confirmed.v1	Order	Order completed	Notification, Fulfillment
order.cancelled.v1	Order	Order cancelled	Inventory, Payment
payment.authorization-required.v1	Payment	Customer action required	Order UI, Notification
payment.authorized.v1	Payment	Funds authorized	Saga
payment.captured.v1	Payment	Funds captured	Saga, Ledger
payment.failed.v1	Payment	Attempt failed	Saga, Notification
payment.refund-succeeded.v1	Payment	Refund confirmed	Ledger, Order
payment.dispute-opened.v1	Payment	Charge disputed	Ledger, Operations
ticket.issued.v1	Ticket	Credential issued	Access, Notification
ticket.revoked.v1	Ticket	Credential invalidated	Access, Audit
access.granted.v1	Access	Gate accepted ticket	Analytics, Security
access.denied.v1	Access	Gate rejected ticket	Security, Analytics
ledger.journal-posted.v1	Ledger	Balanced journal posted	Settlement, Audit
settlement.opened.v1	Ledger	Run snapshot fixed	Operations
settlement.finalized.v1	Ledger	Beneficiary balances finalized	Payout, Analytics
payout.succeeded.v1	Payout	Provider payout completed	Ledger, Notification
payout.failed.v1	Payout	Payout requires attention	Operations
marketplace.stock-reserved.v1	Marketplace	Merch stock held	Order

Event	Owner	Meaning	Consumers
fulfillment.shipment-created.v1	Fulfillment	Shipment accepted	Notification
travel.quotes-received.v1	Travel	Provider quotes normalized	Travel UI
travel.booked.v1	Travel	Reservation confirmed	Ledger, Notification
notification.delivered.v1	Notification	Message delivered	Analytics
notification.failed.v1	Notification	Delivery exhausted	Operations
search.document-updated.v1	Search	Derived document indexed	Search Ops
audit.case-exported.v1	Audit	Evidence bundle ready	Auditor

PART IV

Security, privacy and financial controls

Trust boundaries

- Public client to gateway: untrusted input, rate limits, bot/abuse controls and OIDC validation.
- Gateway to services: workload identity, audience-restricted tokens and propagated correlation.
- Service to broker/store: least-privilege credentials and network policies.
- Payment provider to webhook: TLS, signature verification, raw body handling and replay defense.
- Operator tools: stronger authentication, explicit audit and no shared accounts.

Financial controls

Control	Implementation proof
Exact money	Minor units/exact decimal and currency-specific scale tests
Segregation of duties	Different role approves milestone/payout than creator
Immutability	Posted journals cannot be updated or deleted by application role
Idempotency	Business key and provider key appear in tests and audit
Reconciliation	Automated three-way comparison and exception workflow
Sensitive data	Provider vault; redacted logs; synthetic fixtures
Audit	Actor, tenant, action, target, reason, correlation and timestamp

Threat model highlights

Threat	Example	Mitigation
Spoofing	Forged webhook or service token	Signature/audience validation, mTLS/workload identity where justified
Tampering	Price or beneficiary share changed	Signed/versioned snapshot, authorization and audit
Repudiation	Operator denies manual adjustment	Immutable audit and mandatory reason
Information disclosure	PII in event/log	Minimization, redaction and access control
Denial of service	Ticket drop exhausts DB	Rate limit, queueing, load shedding and capacity plan
Elevation	Club admin accesses federation controls	Tenant-aware policy and negative tests

PART IV

Resilience and observability

Resilience policy

Dependency	Timeout	Retry	Circuit/bulkhead	Fallback
Payment API	Short connect + bounded request	Only safe/idempotent technical failures	Dedicated pool and circuit	Pending/reconcile; never assume failed
Travel provider	Bounded	Backoff+jitter	Per-provider bulkhead	Return partial quotes
Notification	Bounded async	Queue retries then DLQ	Provider-specific circuit	Checkout still succeeds
Search	Tight	At most one	Read bulkhead	Limited database-backed exact lookup
Redis	Very tight	None/one	Separate pool	Safe source-of-truth path
Kafka	Producer budget	Library bounded	Backpressure	Outbox retains committed event

Golden telemetry

- HTTP: rate, errors, duration and saturation, split by route template and outcome.
- Messaging: publish failures, consumer lag, processing duration, retry and DLQ count.
- Saga: active age, state, compensation count and stuck threshold.
- Database: pool saturation, query p95, lock wait, deadlocks and transaction retries.
- Finance: capture-to-journal lag, reconciliation mismatches and unbalanced rejection count.
- Business: hold conversion, oversell attempts rejected, ticket issuance lag and access denial reason.

Initial SLO set

Journey	SLI	Target	Alert principle
Browse offers	Successful requests under 500 ms	99.0% / 30d	Burn-rate, not one spike
Create hold	Accepted/valid rejection under 700 ms	99.0% / 30d	Page if error budget burns fast
Checkout	Orders reach terminal/known-pending state	99.5% / 30d	Alert on stuck saga age
Webhook	Durably acknowledged within 2 s	99.9% / 30d	Alert on signature failures/lag
Settlement	Run completes/reconciles by deadline	99.0% per run	Ticket operator with evidence

PART IV

Testing strategy and architecture fitness

Layer	Purpose	Representative proof
Domain unit	Invariants and state machines	No Spring context; property-based cases for money/schedules
Module integration	Transactions, persistence and events	Testcontainers PostgreSQL; rollback/outbox behavior
Architecture	Dependency and ownership rules	Spring Modulith/ArchUnit and forbidden package access
Contract	HTTP and event compatibility	Provider/consumer tests; schema compatibility
Component	Service with real store/broker	Containerized component and deterministic fakes
Saga	Partial failure and compensation	Matrix of timeout, duplicate, reorder and crash
Performance	Contention and capacity	k6/Gatling peak sale; DB lock metrics
Security	Authorization and common weakness	Negative tenant tests, dependency/secret scans
Recovery	Restore/replay correctness	Database restore, projection rebuild, DLQ replay
E2E	Portfolio narrative	Championship scenario from enrollment to payout

Fitness functions

- Competition Core modules cannot access another module's internal package.
- Extracted services cannot share JPA entities, repositories or migrations.
- Every event has owner, version, schema and compatibility test.
- Every remote client has timeout; retry requires idempotency classification.
- No floating-point type appears in money packages or schemas.
- Every database migration is forward compatible with rolling deployment.
- Every service exports standard resource attributes and correlation.
- Every privileged endpoint has a negative authorization test.

PART IV

Deployment, environments and recovery

Environment ladder

Environment	Purpose	Shape
Developer	Fast feedback	Core + dependencies in Compose; Testcontainers in tests
Integration	Contracts and migrations	Ephemeral namespace per change where affordable
Demo	Portfolio presentation	Stable synthetic dataset; public read access and protected operator UI
Failure lab	Chaos/recovery drills	Isolated environment with fault injection

Kubernetes baseline

- Startup probe protects slow JVM initialization; readiness controls traffic; liveness detects unrecoverable stuck process.
- Requests/limits come from measurement, not copying. HPA follows a meaningful bottleneck and includes stabilization.
- Pod disruption budgets and graceful shutdown protect in-flight work; consumers stop polling before termination.
- Network policies and workload identity reduce lateral movement.

Recovery tiers

Tier	Examples	Recovery approach
Authoritative financial	Ledger, payment	PITR + restore drill + provider reconciliation
Authoritative operational	Competition, inventory, order	Backup/PITR + outbox replay
Derived	Search, analytics read models	Rebuild from events/source exports
Ephemeral	Cache, rate counters	Recreate; safe degradation
Evidence	Contracts, audit exports	Versioned object storage + retention policy

PART V

The 49-sprint campaign

Each sprint is a multi-week checkpoint. The scheduler decomposes it into evidence-bearing work packages without skipping dependencies.

Pacing

Reference capacity: four focused hours per week. A sprint normally spans 3-10 weeks at this pace. A quest may require several sessions and remains pending until its full evidence and acceptance criteria are complete.

Estimation model

Work type	Typical active effort	Rule
Focused reading or review	0.5-1.5 h	Must end in notes, a decision or applied evidence
Modeling or ADR	1-3 h	Research, alternatives, trade-offs and revision included
Contained implementation	2-4 h	Code, tests, debugging and documentation included
Complex vertical slice	4-8 h	Split into 2-4 sessions with partial checkpoints
Large architecture/infra change	> 8 h	Split into several quests; never hide it behind one daily task

Personal calibration

Knowledge band	Examples	Planning multiplier
Familiar	Spring Boot, JPA, Security, MockMvc, Docker	1.0-1.25x
Partially known	Spring Modulith, DDD boundaries, Kafka contracts	1.35-1.60x
New/high-risk	Ledger, Saga failures, K8s, IaC, OTel, DR	1.8-2.2x

Portfolio milestones at four hours per week

Milestone	Central effort	Calendar estimate	What becomes demonstrable
Through S08	172 h	43 weeks	Publishable modular sports-core case study
Through S15	364 h	91 weeks	Financially coherent MVP with ticketing, payments and ledger
Through S21	538 h	135 weeks	Distributed showcase with extraction and orchestrated Saga
Through S48	1,212 h	303 weeks	Full mega-backend and Solution Architect defense

Season map

Season	Sprints	Outcome
T0 Foundations	00-03	Repository, modularity, identity and tenant base
T1 Sports Core	04-08	Competition, eligibility, scheduling and contracts
T2 Payments & Ledger	09-15	Inventory, orders, payments, accounting and settlement
T3 Distribution	16-22	Outbox, Kafka, extraction and Sagas
T4 Ecosystem	23-27	Marketplace, travel, sponsorship, payouts and reconciliation
T5 Data	28-32	CQRS, search, cache, analytics and privacy
T6 Platform	33-42	Security, resilience, observability, cloud and recovery
T7 Portfolio	43-48	E2E proof, docs, demo and architect defense

Sprint 00: Repository, licence, vision and scope

Open a professional repository and define the problem, actors, boundaries and initial decisions.

STATUS

In progress

PACKAGES

5

EFFORT

18 h

AT 4 H/WEEK

5 wk

AREAS

Architecture, Domain, Portfolio

RANGE / CONFIDENCE

14-24 h · Medium

EXIT GATE

Public repository, vision, C4 context, governance and first ADR are versioned and reviewable.

Suggested work sequence

- Q01 · Repository, licence and vision
- Q02 · Glossary and invariants
- Q03 · Drivers and NFRs
- Q04 · C4 and decisions
- Q05 · Governance review

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 01: Spring Boot walking skeleton

Create a reproducible build, health endpoint, first test and minimal CI.

STATUS

Pending

PACKAGES

5

EFFORT

14 h

AT 4 H/WEEK

4 wk

AREAS

Spring, Testing

RANGE / CONFIDENCE

10-18 h · High

EXIT GATE

Application runs locally; tests and pipeline are green.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 02: Verifiable modular monolith

Create bounded modules and forbid illegal dependencies.

STATUS

Pending

PACKAGES

5

EFFORT

18 h

AT 4 H/WEEK

5 wk

AREAS

Architecture, Spring

RANGE / CONFIDENCE

14-24 h · Medium

EXIT GATE

Spring Modulith and architecture tests prove boundaries.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 03: Identity, tenant and base

authorization

STATUS

Pending

PACKAGES

6

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS Security, Spring

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Local login, tenant resolution and negative authorization tests.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

PRIMARY RESOURCE [Spring Framework / Security reference](#)

Sprint 04: Clubs, teams and athletes

Implement core aggregates and invariants.

STATUS

Pending

PACKAGES

6

EFFORT

18 h

AT 4 H/WEEK

5 wk

AREAS Domain, Data

RANGE / CONFIDENCE

14-24 h · High

EXIT GATE

Use-case APIs, validation and domain tests.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 05: Competition lifecycle

Model tournament, season, phases and state transitions.

STATUS

Pending

PACKAGES

6

EFFORT

20 h

AT 4 H/WEEK

5 wk

AREAS Domain, Spring

RANGE / CONFIDENCE

16-28 h · Medium

EXIT GATE

Invalid transitions fail and published rules are versioned.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 06: Transactional eligibility and

enrollment

STATUS

Pending

PACKAGES

7

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS Transactions, Data

RANGE / CONFIDENCE

18-32 h · Medium

EXIT GATE

Constraints and concurrent tests prove correctness.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 07: Scheduling, matches and venues

Generate fixtures and prevent time/venue collisions.

STATUS

Pending

PACKAGES

7

EFFORT

24 h

AT 4 H/WEEK

6 wk**AREAS** Algorithms, Domain**RANGE / CONFIDENCE**

18-32 h · Medium

EXIT GATE

Reproducible schedule, conflicts and paginated API.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 08: Contracts and sponsorship

Version contracts, milestones and approvals.

STATUS

Pending

PACKAGES

6

EFFORT

20 h

AT 4 H/WEEK

5 wk

AREAS Domain, Concurrency

RANGE / CONFIDENCE

16-28 h · Medium

EXIT GATE

Optimistic locking and audit trail proven.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 09: Catalog and ticket offers

Create price books, fees and sellable offers.

STATUS

Pending

PACKAGES

6

EFFORT

18 h

AT 4 H/WEEK

5 wk

AREAS Commerce, Data

RANGE / CONFIDENCE

14-24 h · High

EXIT GATE

Pricing rules and snapshots are tested.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 10: Concurrent inventory holds

Reserve expiring capacity without overselling.

STATUS

Pending

PACKAGES

8

EFFORT

26 h

AT 4 H/WEEK

7 wk

AREAS

Concurrency, Testing

RANGE / CONFIDENCE

20-36 h · Medium

EXIT GATE

Race test shows zero oversell and correct expiry.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 11: Order idempotency

Create replay-safe orders with command fingerprinting.

STATUS

Pending

PACKAGES

7

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS Payments, Data

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Same key is safe; mismatched payload is rejected.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 12: Payment adapter and webhooks

Integrate sandbox PaymentIntents behind a port.

STATUS

Pending

PACKAGES

9

EFFORT

30 h

AT 4 H/WEEK

8 wk

AREAS Payments, Security

RANGE / CONFIDENCE

22-40 h · Medium

EXIT GATE

Signed webhook, duplicate handling and retries pass.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 13: Double-entry ledger

Post immutable, balanced journals.

STATUS

Pending

PACKAGES

10

EFFORT

36 h

AT 4 H/WEEK

9 wk

AREAS

Finance, Transactions

RANGE / CONFIDENCE

28-48 h · Low

EXIT GATE

Every journal balances; trace and reconciliation pass.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 14: Refunds and chargebacks

Model partial/full refund and disputes.

STATUS

Pending

PACKAGES

8

EFFORT

28 h

AT 4 H/WEEK

7 wk

AREAS Payments, Testing

RANGE / CONFIDENCE

22-38 h · Low

EXIT GATE

Order, payment and ledger converge under failure.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 15: Matchday settlement v1

Allocate sale proceeds among beneficiaries.

STATUS

Pending

PACKAGES

10

EFFORT

32 h

AT 4 H/WEEK

8 wk

AREAS Finance, Domain

RANGE / CONFIDENCE

24-44 h · Low

EXIT GATE

Exact arithmetic, audit and repeat-safe run.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 16: Outbox and Inbox

Eliminate dual writes and duplicate effects.

STATUS

Pending

PACKAGES

8

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS Messaging, Data

RANGE / CONFIDENCE

18-32 h · Medium

EXIT GATE

Failure injection loses no event and duplicates no effect.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 17: Kafka and event contracts

Design topics, keys, partitions and schema evolution.

STATUS

Pending

PACKAGES

8

EFFORT

26 h

AT 4 H/WEEK

7 wk

AREAS

Messaging, Infrastructure

RANGE / CONFIDENCE

20-36 h · Medium

EXIT GATE

Compatible contract and ordering tests.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 18: Extract Notification

Separate user communication and retry policy.

STATUS

Pending

PACKAGES

7

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS

Architecture, Messaging

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Independent deploy, DLQ and idempotent consumer.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 19: Extract Ticket Inventory

Apply Strangler and database ownership.

STATUS

Pending

PACKAGES

10

EFFORT

30 h

AT 4 H/WEEK

8 wk

AREAS Architecture, Data

RANGE / CONFIDENCE

22-40 h · Low

EXIT GATE

No cross-service joins; reversible migration and contracts.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 20: Extract Payment and Ledger

Isolate financial risk and release cadence.

STATUS

Pending

PACKAGES

10

EFFORT

32 h

AT 4 H/WEEK

8 wk

AREAS Architecture, Security

RANGE / CONFIDENCE

24-44 h · Low

EXIT GATE

Versioned API/events and independent reconciliation.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 21: Orchestrated Matchday Saga

Coordinate hold, order, payment, ticket, ledger and compensation.

STATUS

Pending

PACKAGES

12

EFFORT

40 h

AT 4 H/WEEK

10 wk

AREAS

Saga, Testing

RANGE / CONFIDENCE

30-54 h · Low

EXIT GATE

Failure matrix proves convergence and idempotency.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence
- Q11 · Load Measure behavior under representative concurrency and Commit
- Q12 · Defend Explain the sprint decision and failure behavior in a Commit

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 22: Choreography for side effects

Fan out analytics, CRM and notification events.

STATUS

Pending

PACKAGES

8

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS

Messaging, Observability

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Independent consumers and measurable lag/DLQ policy.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 23: Internal Spring platform

Publish a BOM, reusable starters and technical support artifacts through Artifactory.

STATUS

Pending

PACKAGES

12

EFFORT

48 h

AT 4 H/WEEK

12 wk

AREAS Architecture, Spring, Messaging, Infrastructure, Testing

RANGE / CONFIDENCE

36-64 h · Low

EXIT GATE

BOM and starter published; two adoptions; integrated test; security and compatibility demonstrated.

Suggested work sequence

- Q01 · Platform ADR
- Q02 · BOM
- Q03 · Auto-configuration
- Q04 · Kafka policy
- Q05 · Override and opt-out
- Q06 · Integrated test
- Q07 · Artifactory publication
- Q08 · Secure consumption
- Q09 · SemVer
- Q10 · Observability
- Q11 · Two adoptions
- Q12 · Release and SBOM

GUARDRAIL

Artifactory token only in build/CI; Kafka runtime secrets remain separate. The starter shares technical mechanisms, never domain meaning. Validate two services with different configurations.

Architect questions

- Starter, template or platform service?
- Which breaking change will consumers accept?
- Who operates and versions the internal product?
- How do you prevent a shared domain kernel?

Sprint 24: Marketplace and stock

Add merchandise stock and fulfillment states.

STATUS

Pending

PACKAGES

8

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS Domain, Data

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Reservation, order and shipment invariants pass.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 25: Travel provider integrations

Normalize quotes/bookings and tolerate provider failure.

STATUS

Pending

PACKAGES

9

EFFORT

26 h

AT 4 H/WEEK

7 wk**AREAS** Adapters, Resilience**RANGE / CONFIDENCE**

20-36 h · Low

EXIT GATE

Timeouts, retry and compensation are tested.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 26: Sponsorship milestones

Release value through approved milestones.

STATUS

Pending

PACKAGES

8

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS Workflow, Finance

RANGE / CONFIDENCE

18-32 h · Low

EXIT GATE

Segregation of duties and audit trail.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 27: Prizes and payouts

Calculate and execute beneficiary payouts.

STATUS

Pending

PACKAGES

9

EFFORT

28 h

AT 4 H/WEEK

7 wk

AREAS Payments, Security

RANGE / CONFIDENCE

22-38 h · Low

EXIT GATE

Validated payee, idempotent payout and ledger trace.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 28: Financial reconciliation

Compare provider, orders and ledger.

STATUS

Pending

PACKAGES

9

EFFORT

28 h

AT 4 H/WEEK

7 wk
AREAS Finance, Operations

RANGE / CONFIDENCE

22-38 h · Low

EXIT GATE

Re-runnable report and exception queue.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 29: CQRS read models

Build rebuildable fan, club and operations views.

STATUS

Pending

PACKAGES

8

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS

CQRS, Messaging

RANGE / CONFIDENCE

18-32 h · Medium

EXIT GATE

Projection replay and lag measurement.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 30: Sports search

Index public entities without becoming authority.

STATUS

Pending

PACKAGES

7

EFFORT

20 h

AT 4 H/WEEK

5 wk**AREAS** Search, Data**RANGE / CONFIDENCE**

16-28 h · Medium

EXIT GATE

Zero-downtime rebuild and visible freshness.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 31: Cache and rate limiting

Add cache-aside and actor quotas safely.

STATUS

Pending

PACKAGES

7

EFFORT

20 h

AT 4 H/WEEK

5 wk

AREAS Redis, Security

RANGE / CONFIDENCE

16-28 h · Medium

EXIT GATE

Invalidation rules and hit/miss/limit tests.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 32: Operational analytics

Feed a columnar model for sales and attendance.

STATUS

Pending

PACKAGES

9

EFFORT

26 h

AT 4 H/WEEK

7 wk

AREAS Analytics, Messaging

RANGE / CONFIDENCE

20-36 h · Low

EXIT GATE

Replayable pipeline and freshness dashboard.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 33: Privacy and lifecycle

Classify PII, retention and deletion by store.

STATUS

Pending

PACKAGES

8

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS Privacy, Data

RANGE / CONFIDENCE

18-32 h · Low

EXIT GATE

Data map and verified retention/deletion jobs.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 34: Fine-grained OIDC/OAuth2

Harden scopes, roles and multi-tenant policy.

STATUS

Pending

PACKAGES

8

EFFORT

24 h

AT 4 H/WEEK

6 wk

AREAS

Security, Spring

RANGE / CONFIDENCE

18-32 h · Medium

EXIT GATE

Negative tests and key/secret rotation evidence.

Suggested work sequence

- Q01 · Frame Write the concrete problem
 - Q02 · Contract Define command
 - Q03 · Domain Implement the smallest domain rule or state transition with Commit
 - Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
 - Q06 · Failure Inject a relevant conflict
 - Q07 · Observe Add logs
 - Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 35: Threat model and secrets

Apply STRIDE and protect sensitive configuration.

STATUS

Pending

PACKAGES

8

EFFORT

22 h

AT 4 H/WEEK

6 wk**AREAS** Security, Infrastructure**RANGE / CONFIDENCE**

16-30 h · Medium

EXIT GATE

Prioritized mitigations and clean secret scan.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 36: Synchronous resilience

Bound timeout/retry/circuit/bulkhead behavior.

STATUS

Pending

PACKAGES

9

EFFORT

26 h

AT 4 H/WEEK

7 wk

AREAS Resilience, Testing

RANGE / CONFIDENCE

20-36 h · Medium

EXIT GATE

Failure tests avoid cascades and respect latency budgets.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 37: Async resilience and replay

Handle poison messages, DLQ and backpressure.

STATUS

Pending

PACKAGES

8

EFFORT

26 h

AT 4 H/WEEK

7 wk

AREAS Messaging, Operations

RANGE / CONFIDENCE

20-36 h · Medium

EXIT GATE

Idempotent replay tool and operator runbook.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 38: OpenTelemetry and SLOs

Correlate traces, metrics and logs across sagas.

STATUS

Pending

PACKAGES

10

EFFORT

30 h

AT 4 H/WEEK

8 wk

AREAS Observability

RANGE / CONFIDENCE

22-40 h · Low

EXIT GATE

End-to-end trace, RED metrics, SLI/SLO and alerts.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 39: Load, concurrency and capacity

Measure p95/p99, contention and saturation.

STATUS

Pending

PACKAGES

10

EFFORT

32 h

AT 4 H/WEEK

8 wk

AREAS Performance, Data

RANGE / CONFIDENCE

24-44 h · Low

EXIT GATE

Reproducible report and two verified improvements.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 40: Containers and Kubernetes

Deploy components with correct probes/resources.

STATUS

Pending

PACKAGES

9

EFFORT

28 h

AT 4 H/WEEK

7 wk
AREAS Infrastructure

RANGE / CONFIDENCE

22-38 h · Low

EXIT GATE

Local cluster, rollout and probe behavior proven.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 41: Infrastructure as Code

Provision network, compute, data, broker and secrets.

STATUS

Pending

PACKAGES

10

EFFORT

34 h

AT 4 H/WEEK

9 wk

AREAS Cloud, Terraform

RANGE / CONFIDENCE

26-46 h · Low

EXIT GATE

Reproducible plan, state boundaries and cost controls.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 42: CI/CD and supply chain

Add quality gates, SBOM, scanning and promotion.

STATUS

Pending

PACKAGES

9

EFFORT

28 h

AT 4 H/WEEK

7 wk**AREAS** Delivery, Security**RANGE / CONFIDENCE**

22-38 h · Medium

EXIT GATE

Signed artifacts, controlled deployment and rollback.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 43: Backup, restore and disaster

recovery

STATUS

Pending

PACKAGES

10

EFFORT

30 h

AT 4 H/WEEK

8 wk**AREAS** Reliability, Data**RANGE / CONFIDENCE**

22-40 h · Low

EXIT GATE

Timed restore exercise and documented gaps.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 44: Championship end-to-end test

Run enrollment through payout with injected failures.

STATUS

Pending

PACKAGES

12

EFFORT

36 h

AT 4 H/WEEK

9 wk

AREAS E2E, Saga

RANGE / CONFIDENCE

28-48 h · Low

EXIT GATE

Automated scenario and consistency evidence.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write
- Q09 · Harden Add security
- Q10 · Close Run the Definition of Done checklist and publish evidence
- Q11 · Load Measure behavior under representative concurrency and Commit
- Q12 · Defend Explain the sprint decision and failure behavior in a Commit

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 45: Cost and performance decisions

Tie architecture to SLO, scale and cost.

STATUS

Pending

PACKAGES

7

EFFORT

20 h

AT 4 H/WEEK

5 wk**AREAS** Architecture, FinOps**RANGE / CONFIDENCE**

16-28 h · Medium

EXIT GATE

Cost model, capacity plan and optimization ADR.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 46: Architecture fitness functions

Automate boundary, contract and security rules.

STATUS

Pending

PACKAGES

7

EFFORT

20 h

AT 4 H/WEEK

5 wk**AREAS** Architecture, Testing**RANGE / CONFIDENCE**

16-28 h · Medium

EXIT GATE

Build fails on each deliberate violation.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 47: OpenAPI, AsyncAPI and C4

Publish living contracts and diagrams.

STATUS

Pending

PACKAGES

8

EFFORT

22 h

AT 4 H/WEEK

6 wk

AREAS

Documentation

RANGE / CONFIDENCE

16-30 h · Medium

EXIT GATE

Navigable, versioned docs consistent with code.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 48: Demo and case study

Show decisions and controlled failures, not only happy path.

STATUS

Pending

PACKAGES

8

EFFORT

18 h

AT 4 H/WEEK

5 wk

AREAS Portfolio

RANGE / CONFIDENCE

14-24 h · High

EXIT GATE

Script/video, dashboards and evidence narrative.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict
- Q07 · Observe Add logs
- Q08 · Decide Write

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 49: Solution Architect defense

Defend trade-offs and future roadmap.

STATUS

Pending

PACKAGES

6

EFFORT

14 h

AT 4 H/WEEK

4 wk

AREAS Portfolio, Architecture

RANGE / CONFIDENCE

10-18 h · High

EXIT GATE

20-minute defense, question bank and retrospective.

Suggested work sequence

- Q01 · Frame Write the concrete problem
- Q02 · Contract Define command
- Q03 · Domain Implement the smallest domain rule or state transition with Commit
- Q04 · Persistence Add migration
- Q05 · Vertical slice Expose one end
- Q06 · Failure Inject a relevant conflict

Architect questions

- Which invariant is authoritative, and where is it enforced?
- Which failure leaves an unknown outcome, and how is it reconciled?
- Which evidence justifies an independent deployment boundary?
- How is it migrated or rolled back without downtime?

Sprint 50: Portfolio and LinkedIn launch

Publish the release, case study, demo and an honest professional narrative.

STATUS

Pending

PACKAGES

7

EFFORT

16 h

AT 4 H/WEEK

4 wk

AREAS Portfolio, Architecture, Testing

RANGE / CONFIDENCE

12-24 h · Medium

EXIT GATE

Release, README, case study, demo, LinkedIn and reviewer journey are verified.

Suggested work sequence

- Q01 · Release
- Q02 · Final README
- Q03 · Case study
- Q04 · Demo and assets
- Q05 · LinkedIn Featured
- Q06 · Launch post
- Q07 · Defence feedback

GUARDRAIL

Never invent scale or production metrics. Separate measured evidence, tests, simulations and professional ambition.

Architect questions

- What can a reviewer verify in ten minutes?
- Which claim is backed by a measurement?
- Which limitation will you publish?
- Which technical conversation should this provoke?

Additional acceptance scenarios

ID	Given / When	Then
AC-026	A service consumes the starter	Valid beans and properties; no shared domain
AC-027	Artifactory token is revoked	Build fails; deployed runtime is unaffected
AC-028	Kafka credentials rotate	Service reloads without republishing starter
AC-029	A local override exists	Service bean visibly wins precedence
AC-030	Starter is disabled	No common beans; service starts with alternative
AC-031	A poison message arrives	Bounded retry, DLT, headers and audited replay
AC-032	Spring baseline changes	Matrix detects incompatibility before release
AC-033	Two services adopt	Shared defaults; different local configuration
AC-034	Configuration breaks	Major version, deprecation and migration guide
AC-035	Reviewer opens release	README and demo work without prior context

ORIGINAL CATALOGUE PRESERVED

AC-001 to AC-025 from blueprint v1 remain valid: contention, payments, webhooks, Saga, ledger, replay, tenant isolation, backup and capacity.

Revised ADR backlog

ADR	Decision	Must include
ADR-000	Repository and licence	Public/private, Apache-2.0, governance
ADR-001	Modular monolith	Boundaries and extraction triggers
ADR-008	Outbox relay	Polling, CDC or event registry
ADR-009	Serialisation	JSON/Avro/Protobuf compatibility
ADR-010	Saga	Orchestration, compensation, reconciliation
ADR-025	Internal platform	Starter vs template vs sidecar/service
ADR-026	BOM and versioning	SemVer, baseline and deprecation
ADR-027	Kafka error policy	Retry, DLT, replay and opt-out
ADR-028	Artifactory auth	Scope, expiry, CI and rotation
ADR-029	Adoption	DevEx metrics and ownership
ADR-030	Portfolio launch	Release, evidence and claims

RULE

An ADR is not a technology essay. It records context, real alternatives, the decision, consequences, reversibility and signals for reconsideration.

Repository structure

Path	Purpose
/README.md	Cover, quick start, architecture and demo
/LICENSE	Apache-2.0
/docs/adr	Numbered decisions
/docs/diagrams	C4, Saga and data diagrams
/contracts	OpenAPI, AsyncAPI and schemas
/apps/competition-core	Modular monolith
/services/*	Extracted deployable units only
/platform-lib/platform-bom	Compatible versions
/platform-lib/kafka-autoconfigure	Beans and properties
/platform-lib/kafka-starter	Consumable starter
/platform-lib/test-support	Technical test utilities
/templates/service-template	Golden path
/platform/{local,k8s,iac}	Environments and infrastructure
/tests/e2e	Scenarios and failures
/runbooks	DLT, Saga, restore and rollback

Internal platform checklist

- A repeated problem is demonstrated in at least two consumers.
- Extraction ADR and explicit boundaries exist.
- BOM and transitive dependencies are minimal.
- @AutoConfiguration, conditions and properties are validated.
- Defaults, override and opt-out are documented.
- One integrated test covers beans, wiring, retry, backoff and DLT.
- Snapshot and release are published with provenance and SBOM.
- Artifactory token exists only in CI/build; runtime secrets remain separate.
- SemVer, compatibility, changelog, deprecation and rollback are explicit.
- Two real adoptions use different configuration.
- DevEx metrics and a feedback channel exist.
- No shared domain types, events or rules leak into the platform library.

RETIREMENT CRITERION

If the starter blocks upgrades, multiplies local exceptions or forces coordinated releases, split it, simplify it, or return the capability to a template.

Additional architecture-defence questions

- 26. Why use a starter instead of copied configuration or a sidecar?
- 27. What belongs in the BOM, and what must each service version itself?
- 28. How does auto-configuration preserve local decisions?
- 29. What does the single @EmbeddedKafka test actually prove?
- 30. Why is the Artifactory token not a runtime secret?
- 31. How do you manage compatibility across Java, Spring Boot and Kafka?
- 32. Which breaking change requires a major version, and how do consumers migrate?
- 33. Which metric proves that the platform improves Developer Experience?
- 34. How do you prevent a common module from becoming a shared domain model?
- 35. Which component would you buy/manage, and which would you build?
- 36. Who operates the internal platform, and what support does it promise?
- 37. What changes with ten consuming teams?
- 38. How would you retire a failed starter without stopping releases?
- 39. Which portfolio claim is simulated, and which is measured?
- 40. What would you show a CTO in five minutes?

Sprint 50 · LinkedIn and portfolio launch

Artifact	Definition of Done
Release	Tag, notes, licence, changelog and security check
README	Problem, drivers, decisions, demo and limits
Case study	Context, alternatives, failures and results
Video	4-7 min: flow plus observable failure
Assets	Readable C4, screenshot and social preview
Featured	Repository/case study featured on profile
Post	Hook, three decisions, difficulty and evidence
CV	Verifiable bullets; no self-awarded title
Feedback	Three external observations and corrections

POST STRUCTURE

Real problem → counter-intuitive decision → three lessons → visual evidence → direct link → technical question.
Avoid buzzword lists and invented scale.

Additional primary sources

[GitHub · Creating a repository](#)

[GitHub · Licensing a repository](#)

[Apache License 2.0](#)

[Spring Boot · Developing auto-configuration](#)

[Spring Kafka · Handling exceptions](#)

[Spring Boot · Externalized configuration](#)

[Maven · Dependency mechanism](#)

[Semantic Versioning 2.0.0](#)

[JFrog · Maven repositories](#)

[JFrog · Access tokens](#)

[LinkedIn · Featured section](#)

[GitHub · Social preview](#)

CLOSING PRINCIPLE

Build a coherent solution that survives duplicates, timeouts, contention and disagreement. The portfolio becomes senior when every pattern protects an invariant and every boundary has evidence.